

8.9 VIRTUAL BASE CLASSES ✓

We have just discussed a situation which would require the use of both the multiple and multilevel inheritance. Consider a situation where all the three kinds of inheritance, namely, multilevel, multiple and hierarchical inheritance, are involved. This is illustrated in Fig. 8.12. The 'child' has two direct base classes 'parent1' and 'parent2' which themselves have a common base class 'grandparent'. The 'child' inherits the traits of 'grandparent' via two separate paths. It can also inherit directly as shown by the broken line. The 'grandparent' is sometimes referred to as *indirect base class*.

Inheritance by the 'child' as shown in Fig. 8.12 might pose some problems. All the public and protected members of 'grandparent' are inherited into 'child' twice, first via 'parent1' and again via 'parent2'. This means, 'child' would have *duplicate* sets of the members inherited from 'grandparent'. This introduces ambiguity and should be avoided.

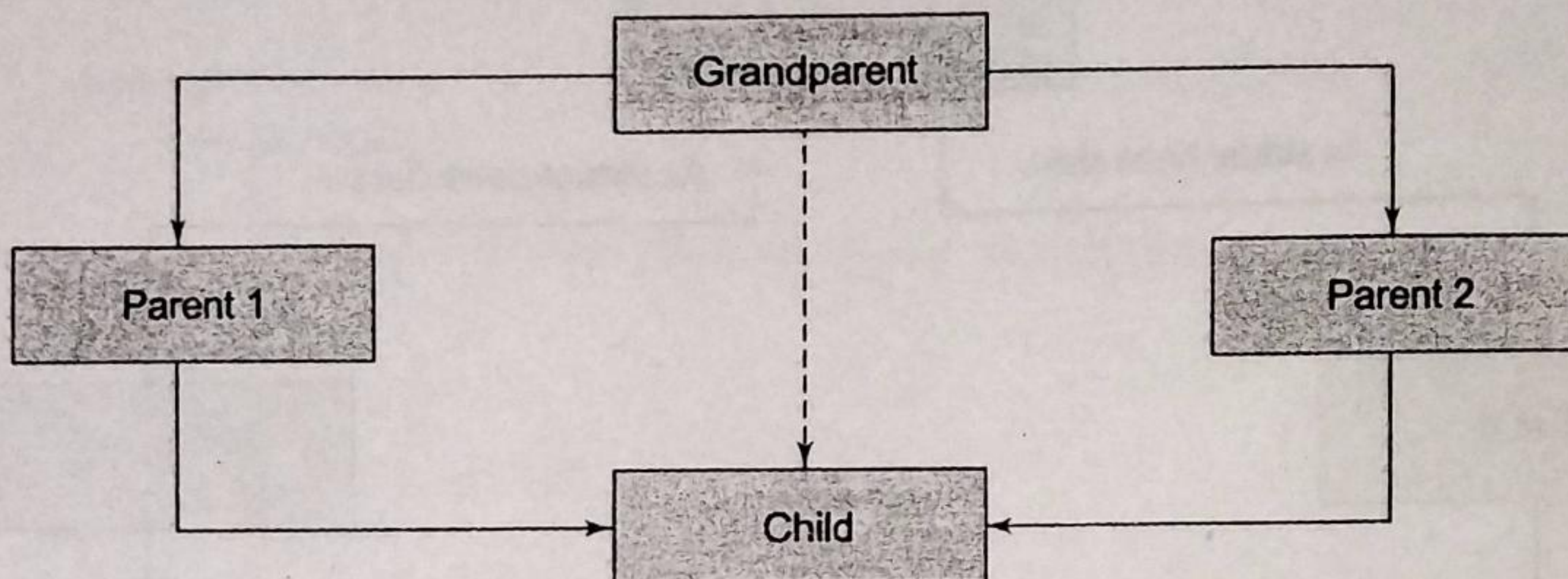


Fig. 8.12 *Multipath inheritance*

The duplication of inherited members due to these multiple paths can be avoided by making the common base class (ancestor class) as *virtual base class* while declaring the direct or intermediate base classes as shown below:

```

class A                                // grandparent
{
    .....
    .....
};
class B1 : virtual public A             // parent1
{
    .....
    .....
};
class B2 : public virtual A            // parent2
{
    .....
    .....
};
class C : public B1, public B2         // child
{
    .....
    .....
};
// only one copy of A
// will be inherited
  
```

When a class is made a **virtual base class**, C++ takes necessary care to see that only one copy of that class is inherited, regardless of how many inheritance paths exist between the virtual base class and a derived class.

Note The keywords *virtual* and *public* may be used in either order.

For example, consider again the student results processing system discussed in Sec. 8.8. Assume that the class **sports** derives the **roll_number** from the class **student**. Then, the inheritance relationship will be as shown in Fig. 8.13.

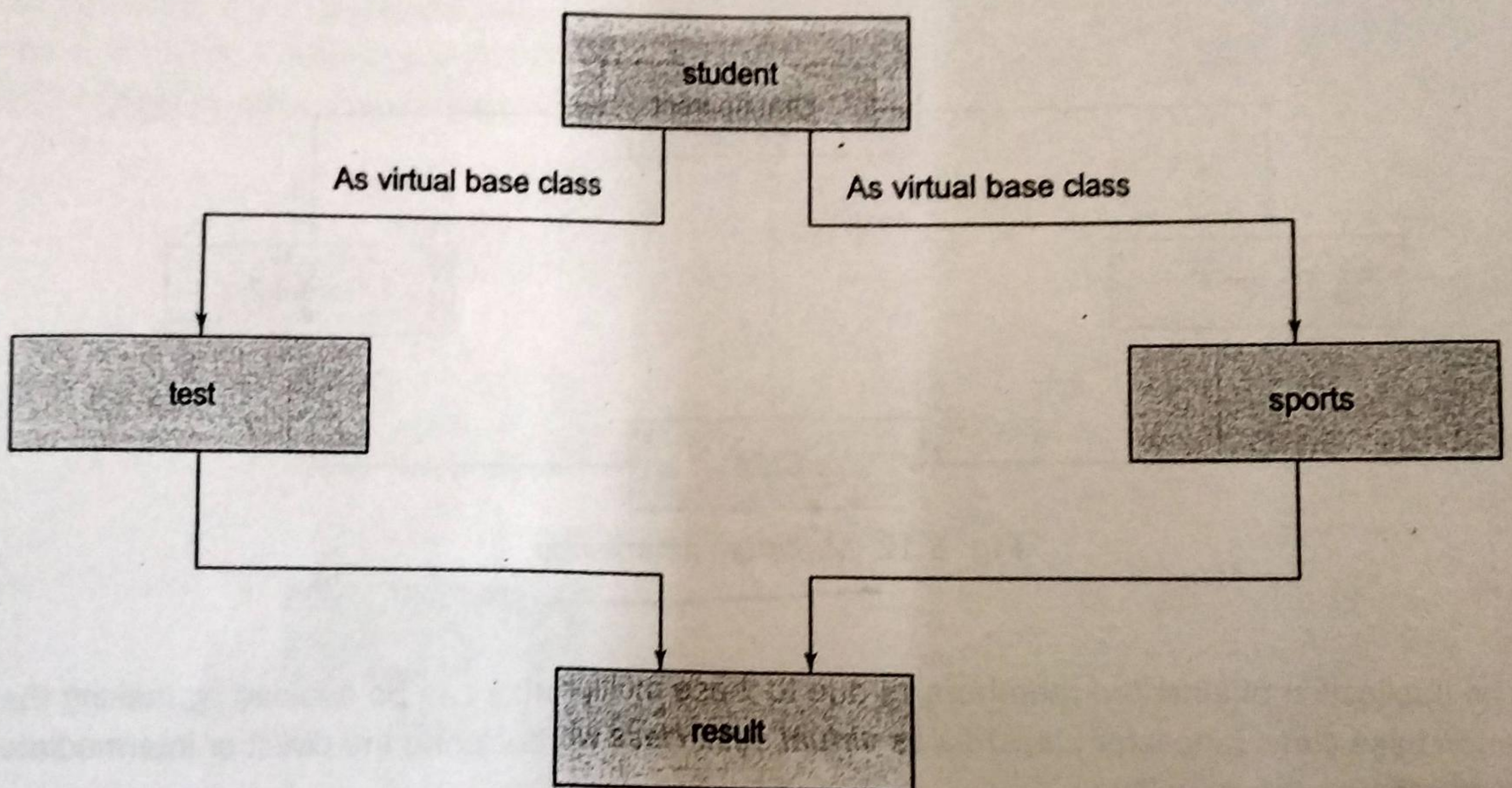


Fig. 8.13 Virtual base class

A program to implement the concept of virtual base class is illustrated in Program 8.6.

Program 8.6 Virtual Base Class

```
#include <iostream>

using namespace std;

class student
{
    protected:
        int roll_number;
    public:
        void get_number(int a)
        {
            roll_number = a;
        }
        void put_number(void)
        {
            cout << "Roll No: " << roll_number << "\n";
        }
};

class test : virtual public student
{
    protected:
```



```

        float part1, part2;
    public:
        void get_marks(float x, float y)
        {
            part1 = x; part2 = y;
        }
        void put_marks(void)
        {
            cout << "Marks obtained: " << "\n"
                << "Part1 = " << part1 << "\n"
                << "Part2 = " << part2 << "\n";
        }
};

class sports : public virtual student
{
    protected:
        float score;
    public:
        void get_score(float s)
        {
            score = s;
        }
        void put_score(void)
        {
            cout << "Sports wt: " << score << "\n\n";
        }
};

class result : public test, public sports
{
    float total;
    public:
        void display(void);
};

void result :: display(void)
{
    total = part1 + part2 + score;

    put_number();
    put_marks();
    put_score();

    cout << "Total Score: " << total << "\n";
}

int main()
{
    result student_1;
    student_1.get_number(678);
    student_1.get_marks(30.5, 25.5);
    student_1.get_score(7.0);
    student_1.display();

    return 0;
}

```


The output of Program 8.6 would be:

Roll No: 678

Marks obtained:

Part1 = 30.5

Part2 = 25.5

Sport wt: 7

Total Score: 63

(In the above example, only one copy of **roll_number** variable is inherited by the result class from the student class. Thus, any changes made to **roll_number** by any of the derived classes are made to this single instance. This ensures data integrity.)